



Job Market Data Pipeline — Saudi Arabia

Project Proposal (updated)

Team A

Name	Email	Cohort
Rimaz Khalid Alghamdi	rimazalghamdi1@gmail.com	Cohort 1 Online
Ghadah BaniAli	Ghadahbaniali@gmail.com	
Azizah Alharbi	Azizaal7rby@gmail.com	
Shahd Aldukhayil	ShahdAldukhayil@gmail.com	

Supervised by:

Original proposal submitted: 3 Sep 2026

Updated to the delivered project: 24 Sep 2026

Table of Contents

What Changed Since the Proposal	3
1 Problem	4
2 Project Objective.....	4
3 Target Market.....	5
4 Data Sources	5
5 Pipeline Architecture	6
6 Technology Stack.....	9
7 Risks & Mitigation.....	9
8 Roadmap & Deliverables	10
9 Expected Output & Downstream Applications.....	11

What Changed Since the Proposal

The proposal of 3 September described the plan before any source had been tested. This version keeps its structure and records what the team built. Each change came from testing the sources, the bootcamp stack, or the data itself. The table lists every section that changed; the sections after it give the full, updated content.

Section	Proposal (3 Sep)	Delivered project (29 Sep)	Reason
4 Data sources	6 ATS (Greenhouse, Lever, Ashby, Workable, SmartRecruiters, Recrutee) + Jooble	4 ATS (Ashby, Greenhouse, SmartRecruiters, Workable) + 2 aggregators (Jooble, JSearch)	Lever and Recrutee returned very few Saudi postings through their APIs (14 and 17) and still reach the dataset through Jooble. JSearch added full descriptions and LinkedIn-sourced jobs through Google for Jobs
5.1 Architecture	Raw → Staging → Curated → Presentation	ADLS raw → Snowflake RAW → dbt staging → intermediate → MARTS → Power BI and ADLS curated/	The course stack (ADLS, Snowflake, dbt, Power BI); an intermediate layer holds matching and lifecycle logic
5.2 Deduplication	Blocking + semantic matching (text embeddings) + threshold	Publisher rule + blocking on company and city + exact key match, then a fuzzy tier: titles merged when their Jaccard word overlap reaches 80, a threshold chosen from 88 labelled pairs	Explainable, testable rules in SQL; every merge can be traced to its key
5.2 Idempotency	SHA-256 business key, MERGE loading, watermark	Hashed surrogate keys, COPY INTO load tracking, full dbt rebuild from RAW, run_id per run	COPY INTO never loads a file twice; rebuilding from immutable RAW replaces MERGE and watermarks
5.2 Lifecycle	first_seen, last_seen, consecutive_misses, disappeared	Six date roles (posted, first seen, last seen, opening, open until, disappeared) and status open / disappeared / unknown	Status comes from employer boards, where a missing posting means the job left the board
6 Technology	requests, Scrappy, Selenium/Playwright; local JSON; Pandas; Docker DB	Python requests; ADLS Gen2; Snowflake; dbt 1.12 with 288 tests; Power BI; GitHub	All six sources offer APIs, so no scraping was needed; the stack follows the capstone guide
8 Roadmap	Five phases, weeks 3–6	Delivered on schedule, with repeated weekly collections	Repeated pulls make the job lifecycle measurable
9 Output	One flat table of 11 columns	Star schema: fct_jobs + 8 dimensions + bridge_job_skill (10 tables)	Every proposed column is kept, and skills, lifecycle, salary and source overlap are added

1 Problem

Job market data is available across multiple platforms and sources, but the data is often fragmented and inconsistent. Job postings may differ in:

- Structure and field names
- Job title and location formats
- Salary representation
- Required skills and qualifications
- Completeness and freshness
- Duplicate postings across different sources

As a result, raw job posting data cannot be directly used for reliable downstream analysis or applications without significant data processing and quality validation. The challenge is to build a repeatable data pipeline that transforms fragmented job postings into a standardized, reliable, and analysis-ready dataset.

2 Project Objective

To build a repeatable Data Engineering pipeline that transforms fragmented job postings into a reliable, standardized, and curated dataset. The project aims to:

1. Identify and evaluate feasible job data sources for the selected market.
2. Collect and ingest job posting data from selected sources.
3. Explore and understand the structure and quality of raw job data.
4. Clean and standardize job posting information across different sources.
5. Handle data quality issues including missing, duplicate, inconsistent, and outdated records.
6. Build a repeatable data pipeline for ingestion and transformation.
7. Produce a curated and reliable dataset that can support downstream applications and analysis.

Business questions. The curated dataset is modelled to answer nine questions about the Saudi job market:

#	Question
Q1	How many unique job openings were open in Saudi Arabia during the observation period?
Q2	Which regions had the most open job openings? City is the drill-down.
Q3	Which employers had the most open job openings, excluding undisclosed employers and recruitment agencies?
Q4	Which role families had the most open job openings? Job category is the drill-down.
Q5	What share of open job openings falls into each employment type?
Q6	Which experience levels are most requested in each region?
Q7	How many new job openings appeared in each week?
Q8	For job openings that disappeared in each week, what was the median number of days they were listed?

#	Question
Q9	Which skills are mentioned by the largest share of open job openings that have a full description?

3 Target Market

Target market: Saudi Arabia. Why Saudi Arabia?

- A rapidly evolving labor market with opportunities across public and private sectors.
- Job opportunities are distributed across multiple platforms and sources.
- Job seekers rely heavily on digital employment platforms.
- Saudi Arabia has a national unified employment platform, Jadarat, covering job opportunities across public and private sectors.
- This makes Saudi Arabia a relevant market for building a localized and standardized job-market dataset.

Every source is filtered to postings located in Saudi Arabia, and locations are standardised to the Saudi country → region → city hierarchy.

4 Data Sources

4.1 Collected sources

Source	Platform	Access type	HTTP	Why it was selected
Ashby	ATS	Public API	GET	Real posting date, structured address, remote flag
Greenhouse	ATS	Public API	GET	Company name and real posting date in every record
SmartRecruiters	ATS	Public API	GET	Server-side Saudi filter (country=sa); richest structured location; experience level and industry
Workable	ATS	Public widget API	GET	High Saudi share; experience level, industry, employment type
Jooble	Job aggregator	Authenticated API (key)	POST	Largest volume; reaches platforms outside the team's ATS coverage
JSearch (OpenWeb Ninja)	Job aggregator	Authenticated API (key)	GET	Full descriptions, publisher field and employment type; surfaces LinkedIn and other boards through Google for Jobs

Employer boards (ATS) are pulled in full on each collection, so a posting that is no longer on its board has left the market. Aggregators are collected through a fixed matrix of city, keyword and date-window queries that is repeated every week.

4.2 Tested, not collected

Source	Result of testing
Lever	Only 3 of 9 tested boards answer through the API, for 14 Saudi postings. Lever postings reach the dataset through Jooble.
Recruitee	One board tested, 17 Saudi postings. Recruitee postings reach the dataset through Jooble.

4.3 Excluded sources

Source	Reason for exclusion
LinkedIn, Indeed, GulfTalent, NaukriGulf, Glassdoor, Bayt	Terms of use and robots.txt explicitly prohibit automated data collection
Jadarat, Taqat, Qiwa	Require authentication through Nafath (Saudi national SSO)

5 Pipeline Architecture

5.1 Data Flow Diagram – DFD

The pipeline is ELT: raw data lands untouched and every transformation runs inside Snowflake with dbt. One command (pipeline/run_pipeline.py) runs the steps extract → upload → load → freshness → build → export, gives the run a run_id, and stops at the first failed step, so the curated export only receives a build that passed every blocking test.

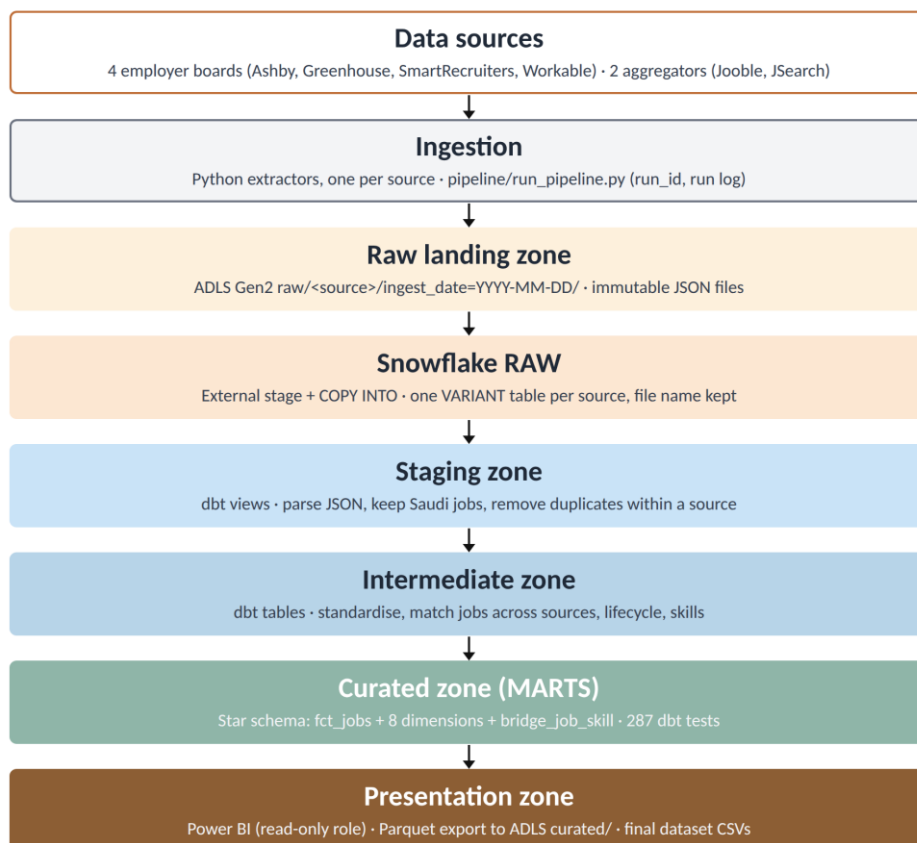


Figure 1 Data flow of the delivered pipeline

5.2 Pipeline Mechanics & Transformation Protocols

A. Data & record validation rules

- **File-level check:** every aggregator page keeps its HTTP status and raw response; only pages with status 200 and valid JSON are read. dbt source freshness checks that each source was loaded recently.
- **Required fields check:** dbt tests (not_null, unique, relationships, accepted_values) on every key and coded field, and a reconciliation test that RAW rows equal staging rows plus the rows removed in scope and dedup.

B. Standardization logic (rules kept in dbt seeds, so they can be reviewed and changed without code)

- **Geography hierarchy:** Arabic and English spellings mapped to a standard city and region (seed_city_mapping); country → region → city.
- **Role normalization:** title keywords mapped to a job category and role family (seed_job_categories, seed_role_families).
- **Employment type normalization:** source values mapped to fixed categories (Full-time, Part-time, Contract, Internship, and others).
- **Seniority normalization:** the source field first; where it is missing, title rules (seed_seniority_keywords). Each value records its basis (source or title).
- **Work arrangement normalization:** workplace type and remote status derived together from the source fields.
- **Salary standardization:** parsed from the published text (currency, range, period) and converted to SAR per month; USD at 3.75 SAR.

C. Deduplication & entity resolution

- **Within a source:** the same posting seen in several pulls or several queries is kept once in staging (32,418 copies removed in the final build).
- **Across sources:** a publisher rule (two postings from the same publisher are never merged), blocking on company and city, and an exact match on normalized title, company and city. Candidate pairs with different wording are scored with Jaccard and Jaro-Winkler similarity; pairs with a Jaccard score of 80 or more are merged. The threshold was chosen from 88 labelled pairs (precision 0.83–0.92), and every fuzzy merge was reviewed by hand.
- **Survivorship:** one representative listing per job by source priority (Workable → SmartRecruiters → Ashby → Greenhouse → JSearch → Jooble); each field then follows its own rule, for example the longest description and the earliest employer posting date.
- **Employer entity resolution:** company names normalized and mapped through seed_company_aliases, which also flags recruitment agencies.

D. Skill & technology extraction

- Descriptions are matched against a skills dictionary (seed_skills: 133 keywords for 125 skills in 12 groups). Results go to bridge_job_skill, one row per job per skill.

E. Idempotency & repeatability

- Raw files in ADLS are never changed or deleted; each collection writes a new ingest_date folder.
- COPY INTO tracks the files already loaded, so a re-run never duplicates RAW rows.
- Every dbt layer is rebuilt from RAW on each run, and surrogate keys are hashes of business keys, so the same input always gives the same keys.
- Each run has a run_id and a row in the run log; the curated export is written per run.

F. Lifecycle & freshness tracking

- Six date roles per job: posted, first seen, last seen, opening, open until and disappeared.
- Status open / disappeared comes from employer boards, which are pulled in full: a job missing from the latest pull of its board has disappeared. Aggregator-only jobs carry status unknown.
- The first pull of each source is the baseline; a job first seen after it is a new opening.

6 Technology Stack

Layer	Tool(s)	Justification
Collection (ingestion)	Python, requests, python-dotenv	All six sources expose APIs, so collection uses documented endpoints only; keys are read from a .env file that is never committed
Orchestration	pipeline/run_pipeline.py	One command runs every step in order, logs each run and stops at the first failure
Raw storage	Azure Data Lake Storage Gen2 (container raw)	Immutable JSON by source and collection date, so any build can be reproduced without calling the APIs again
Warehouse	Snowflake (RAW, STAGING, INTERMEDIATE, MARTS schemas)	External stage and COPY INTO for loading; separate schemas per layer; a read-only role for reporting
Transformation & quality	dbt 1.12 (SQL), dbt_utils, seeds	Version-controlled SQL models, 288 automated tests, documented lineage
Curated storage	Snowflake MARTS; Parquet export to ADLS curated/	Star schema for analysis; a file copy of every table per run
Presentation	Power BI	Dashboard on the MARTS schema only, through the JOB_PIPELINE_REPORTER role
Version control	GitHub	Code, seeds, documentation and analyses shared by the team

7 Risks & Mitigation

The six risks of the proposal are kept with their original severity and matrix. The last column records how each one was handled in the delivered pipeline.

	Negligible	Minor	Moderate	Significant	Severe
Very likely		3	6	4	
Likely					1
Possible			2 5		
Unlikely					
Very unlikely					

Figure 2 Risk matrix (from the proposal)

Risk	Severity	Planned mitigation	Outcome
1 Anti-scraping and access restrictions	High	Prefer official APIs; polite collection; pivot to accessible sources	No scraping was needed: all six sources are APIs. Requests are paced per board, retried on errors, and the aggregator runs stop cleanly on quota limits and resume by query name
2 Schema and layout changes	Medium	Land raw data first; defensive parsing	Raw JSON is stored untouched and parsed in dbt; source tests flag a changed field on the next build
3 Variable data quality	Medium	Standard defaults; null checks; infer from text	Missing values map to an Unknown member in each dimension; experience level is inferred from titles where the source has none; completeness is measured per field
4 Cross-platform duplicates	High	Blocking and semantic matching	Publisher rule, blocking and exact key matching; 20,490 listings resolved to 19,148 jobs, 1,008 of them found on more than one source
5 Source feasibility failure	Medium	Broad pool, then empirical selection	Eight APIs tested; six kept on measured volume and field coverage (section 4)
6 Limited API availability	Medium	Check for APIs before scraping	Every selected source has an API; sites without one were excluded

8 Roadmap & Deliverables

Week	Phase	Task	Result
3 (6–12 Sep)	1. Data ingestion	Test eight APIs; build extractors; first pulls of Jooble, JSearch and Greenhouse	Raw data collected
4 (13–19 Sep)	2. Landing and cleaning	ADLS landing, Snowflake RAW, dbt staging with scope filter and dedup; first pulls of Ashby, Workable and SmartRecruiters	Clean staging layer
4–5	3. Schema design	Business questions, grain, star schema and matching specification	Final schema (data_model.md)
5 (20–26 Sep)	4. Quality testing	Intermediate and marts models; 288 dbt tests; matching threshold chosen from labelled pairs; reconciliation checks; second weekly pulls	Validated dataset
6 (27–29 Sep)	5. Final delivery	Repeat every query and board; final run; Parquet export; Power BI; report and presentation	Final job dataset

Final run (29 Sep 2026, data up to 28 Sep): 52,937 RAW rows in 3,007 files → 20,490 listings → 19,148 unique jobs from 3,004 companies. 323 checks passed with no warnings and no errors, and the 19 layer-reconciliation checks all returned OK.

9 Expected Output & Downstream Applications

9.1 Output

The output is a star schema in the Snowflake MARTS schema: one fact table at the grain of one job, eight dimensions and one bridge for skills. It is also exported as Parquet to ADLS curated/ and as CSV files in final_datasets/.

Star schema: one fact table, eight dimensions and a skills bridge

Job Market Data Pipeline (Saudi Arabia) - marts layer (Snowflake schema MARTS)

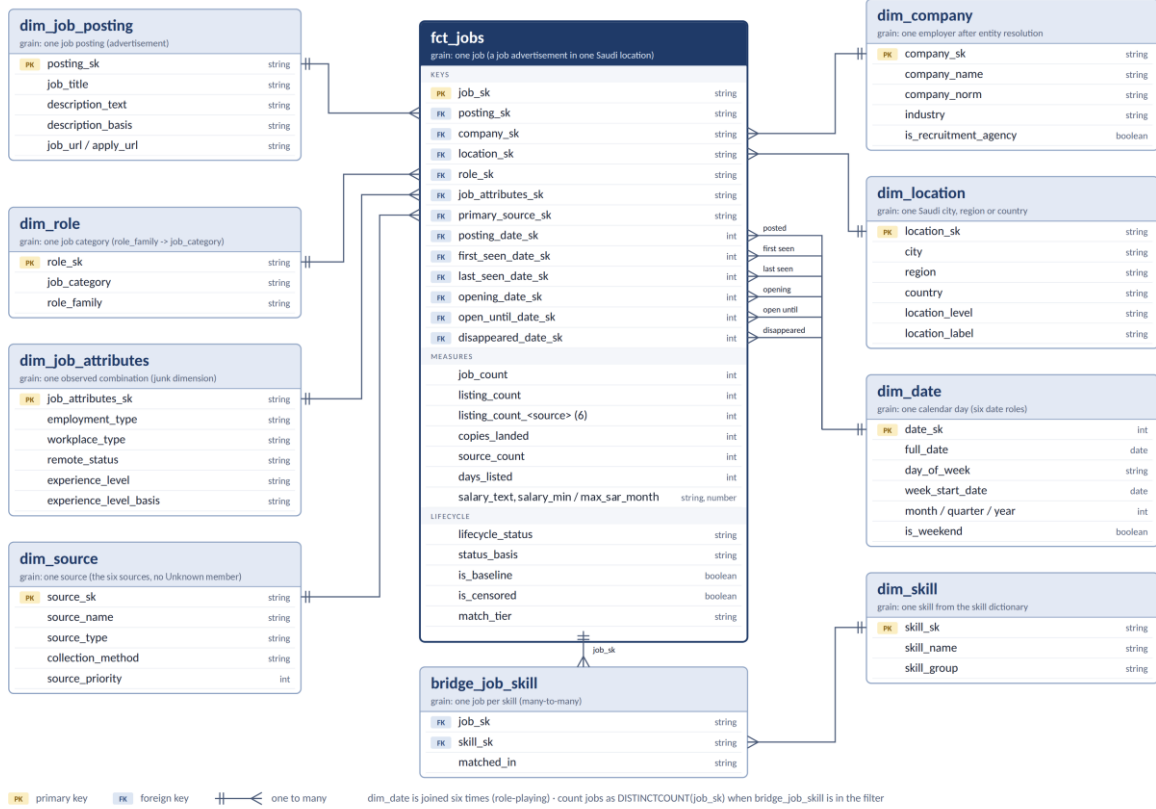


Figure 3 Star schema of the curated dataset

Table	Rows	Content
fct_jobs	19,148	One row per job: measures, lifecycle and six date roles
bridge_job_skill	22,822	One row per job per skill
dim_job_posting	18,694	Title, description and URLs
dim_company	3,005	Employer after entity resolution, industry, recruitment-agency flag
dim_date	3,003	Calendar with week, month, quarter and year
dim_skill	126	Skill and skill group
dim_job_attributes	83	Employment type, workplace type, remote status, experience level
dim_location	58	City, region and country
dim_role	20	Job category and role family
dim_source	6	The six sources, their type and priority

Where each proposed column lives. Every column of the proposal's output table is part of the final model:

Proposed column	Final location	Handling
Job ID	fct_jobs.job_sk; dim_job_posting.posting_sk	Source identifier kept per listing; one hashed key per job after matching
Source Platform	dim_source	Representative source, plus a listing count per source in fct_jobs
Job Title	dim_job_posting.job_title	Cleaned; a normalized key is used for matching
Company	dim_company.company_name	Resolved through the alias seed
Sector	dim_company.industry	Most frequent value across the company's listings
Region & City	dim_location	Country → region → city hierarchy
Employment Type	dim_job_attributes.employment_type	Fixed categories; remote status kept as its own attribute
Required Level	dim_job_attributes.experience_level	Source field or title rule, with its basis
Job Description	dim_job_posting.description_text	Longest cleaned description; source of the skills
Posting Date	fct_jobs.posting_date_sk → dim_date	Converted to Asia/Riyadh; five further date roles added
Job URL	dim_job_posting.job_url, apply_url	From the representative listing

Added beyond the proposal: skills (dim_skill and bridge_job_skill), job lifecycle and new openings, days listed, salary in SAR per month, and source overlap (source_count, listing counts per source).

9.2 Downstream Applications

1. **One job database:** job postings from six sources in one place, with duplicates merged.
2. **Job market insights:** where the jobs are, who is hiring, which roles and experience levels are in demand, and which skills employers ask for.
3. **Job market analysis:** a Power BI dashboard on the MARTS schema that answers the nine business questions, and exported files for further analysis.